Binary Search notes by Aman

1. Breadth-First Search (BFS) / Level Order Traversal

This pattern processes the tree level-by-level. It almost always uses a **Queue** data structure.

- **Standard Level Order:** Printing nodes row by row.
- **Level Averages/Sums:** Calculating the average or sum of values at each depth.
- **Zigzag Traversal:** Processing levels in alternating left-to-right and right-to-left order.
- **Connect Level Siblings:** Setting `next` pointers to point to the node on the right in the same level.
- **Minimum Depth:** Finding the shortest path to a leaf node (often faster than DFS for this specific check).

2. Depth-First Search (DFS)

This pattern explores as far as possible along each branch before backtracking. It uses a **Stack** (iterative) or the **Call Stack** (recursive).

- **Pre-order Traversal (Root-Left-Right):** Useful for cloning trees or serializing structure.
- **In-order Traversal (Left-Root-Right):** specifically vital for Binary Search Trees (BST) as it processes nodes in sorted order.
- **Post-order Traversal (Left-Right-Root):** Useful for deleting nodes (process children before parent) or evaluating mathematical expression trees.

3. Bottom-Up DFS (Post-Order Return)

In this pattern, you calculate answers for the children first and pass information *up* to the parent. This is essentially "Divide and Conquer."

- **Height/Depth Calculation:** The height of a node is `max(left_height, right_height) + 1`.
- **Diameter of Tree:** Finding the longest path between any two nodes. This often requires calculating the height of left/right subtrees at every node.
- **Maximum Path Sum:** Similar to diameter, but calculating the max sum of node values from any node to any node.
- **Lowest Common Ancestor (LCA):** Bubbling up a boolean or node reference if a target is found in the subtrees.

4. Top-Down DFS (Path & Accumulator)

In this pattern, you pass a value *down* from the parent to the children. You often carry an "accumulator" variable or a "current path" list.

- **Root-to-Leaf Path Sum:** Checking if a path exists with a specific sum.
- **Count Paths for a Sum:** Counting how many paths equal a target value (often uses a prefix sum map).
- **Collecting All Paths:** Generating a list of all paths from root to leaf (e.g., "1->2->3").
- **Valid Sequence:** Checking if a path matches a specific sequence of numbers.

5. Tree Structure & Construction

These patterns involve modifying the links or building a new tree from data.

- **Construct Tree from Traversals:** Rebuilding a tree given In-order + Pre-order (or Post-order) arrays.
- **Serialization/Deserialization:** Converting a tree to a string (or array) and back.
- **Invert/Flip Tree:** Swapping left and right children recursively.
- **Flatten Tree to Linked List:** Rearranging pointers to make the tree look like a linear list.

6. Tree View Patterns

These problems ask you to identify what is visible from a specific angle.

- **Right/Left Side View:** Often solved using BFS (taking the last/first element of the level) or DFS (prioritizing one side and tracking depth).
- **Top/Bottom View:** Requires tracking the "horizontal distance" from the root (going left subtracts 1, going right adds 1) often using a Map.

7. Binary Search Tree (BST) Specific Patterns

These only apply when the tree is sorted.

- **Range Search:** pruning branches that don't fall within a query range (e.g., `[low, high]`).
- **Kth Smallest Element:** Using In-order traversal to find the k-th element.
- **Validate BST:** Ensuring every node respects the `Left < Node < Right` rule (often passed down as `(min, max)` constraints).

This is a crucial pattern. In interview terms, **Bottom-Up DFS** is the go-to strategy whenever a problem implies: *"To know the answer for the current node, I first need to know the answer for its left and right children."*

It essentially relies on **Post-order Traversal** (Left → Right → Root).

The Core Concept

In Bottom-Up DFS, information flows from the leaves up to the root.

1. **Base Case:** If the node is `null`, return a neutral value (like 0, null, or 0).
2. **Recursive Step:** Call the function on `node.left` and `node.right`.
3. **Logic:** Do calculation using the results from step 2.
4. **Return:** Pass a result up to the *parent*.

The "Global Variable" Trick: Often, the value you need to **return** to the parent is different from the value you are trying to **calculate** for the final answer.

- *Example:* To find the "Diameter" (widest part of the tree), your function returns the **height** to the parent, but updates a global `maxDiameter` variable along the way.

Pattern 1: Breadth-First Search (BFS)

1. Why do we use it?

We use BFS when the problem requires processing the tree **horizontally** (row by row) rather than vertically (branch by branch).

- **Layer-based Logic:** If you need to calculate averages, maximums, or sums specifically for a "level" or "depth," BFS is the only efficient way to group those nodes together.
- **Shortest Path/Nearest Node:** In a tree, BFS is superior for finding the "closest" node to the root (e.g., the nearest leaf node). Because BFS expands layer by layer, the first time you hit a target, you are guaranteed to have found the shortest path to it. DFS would waste time going deep down the wrong branch first.

2. How to identify this pattern?

Look for these specific keywords or requirements in the problem statement:

- "Traverse level by level"
- "Print the tree row by row"
- "Find the average/sum/max of each level"
- "Connect nodes at the same level" (Next Right Pointers)
- "Zigzag traversal"
- "Minimum depth" or "Shortest path to a leaf"

3. The Core Mechanism

This pattern **always** relies on a **Queue** data structure (First-In-First-Out).

1. **Initialize:** Put the `root` into the Queue.
2. **Loop:** Keep going while the Queue is not empty.
3. **The "Level Size" Trick:** Inside the loop, immediately calculate `int size = queue.size()`. This number tells you exactly how many nodes constitute the *current* level.
4. **Process Level:** Run a `for` loop exactly `size` times. In this loop, you pop a node, process it, and add its children to the back of the queue.

5. Example Questions & Logic Variations

Here is how you tweak the template for common interview questions:

A. Binary Tree Zigzag Level Order Traversal

- **Goal:** Level 0: Left → Right, Level 1: Right → Left, Level 2: Left → Right...
- **Tweak:** Keep a boolean flag `reverse`. If `true`, add items to the *front* of the `currentLevel` list (or reverse the list before adding to `result`). Toggle the flag at the end of the `while` loop.

B. Binary Tree Right Side View

- **Goal:** Return only the nodes visible if you look at the tree from the right.
- **Tweak:** Inside the `for` loop, check `if (i == levelSize - 1)`. This condition means you are currently holding the **last node** of that level. Add only that node to your result.

C. Minimum Depth of Binary Tree

- **Goal:** Find the shortest path from root to a leaf.
- **Tweak:** Inside the `for` loop, check:

Pattern 2: Depth-First Search (DFS)

1. Why do we use it?

DFS is used when we need to explore a branch as deep as possible before backtracking. It is fundamentally about **ordering**—controlling exactly when we visit a parent relative to its children.

- **Exhaustive Search:** When you need to check *every* node or find *all* paths.
- **BST Operations:** "In-order" traversal is the only way to process a Binary Search Tree (BST) in sorted order.
- **Serialization:** "Pre-order" is excellent for saving a tree structure to a file so it can be rebuilt later.

2. How to identify this pattern?

Look for these keywords or requirements:

- "Find all paths from root to leaf"
- "Retrieve elements in sorted order" (indicates In-order DFS on a BST)
- "Flatten the tree"
- "Pre-order", "In-order", or "Post-order" is explicitly mentioned.
- "Validate if a tree is a BST"

3. The Core Mechanism

This pattern relies on a **Stack** data structure. However, in 90% of interview solutions, we use the **Call Stack** via **Recursion** because it is cleaner and easier to write.

There are three specific variations based on *when* you process the "Root" (the current node):

1. **Pre-order (Root → Left → Right):**
   - *Logic:* "Process me, then process my left child, then my right child."
   - *Use case:* Cloning a tree, counting nodes, printing directory structure.
2. **In-order (Left → Root → Right):**
   - *Logic:* "Process left child, then process me, then process right child."
   - *Use case:* **BSTs!** This gives you values in strictly increasing order $(2, 5, 10, 15...)$.
3. **Post-order (Left → Right → Root):**
   - *Logic:* "Process children first, then process me."
   - *Use case:* Deleting a tree (delete children before parent), Bottom-up calculations (covered more in Pattern #3).

```java
public class Solution {
    public void dfs(TreeNode node) {
        // 1. Base Case: If node is null, stop recursion.
        if (node == null) {
            return;
        }

        // --- PRE-ORDER LOGIC HERE ---
        // System.out.println(node.val);
        // ----------------------------

        // 2. Visit Left Child
        dfs(node.left);

        // --- IN-ORDER LOGIC HERE ---
        // System.out.println(node.val);
        // ---------------------------

        // 3. Visit Right Child
        dfs(node.right);

        // --- POST-ORDER LOGIC HERE ---
        // System.out.println(node.val);
        // -----------------------------
    }
}
```

5. Example Questions & Logic Variations

A. Validate Binary Search Tree (BST)

- **Pattern: In-order DFS**
- **Logic:** If you traverse a valid BST in-order, the values *must* be sorted.
- **Tweak:** Keep a global variable `prev`. In the "In-order" spot, check if `node.val <= prev`. If it is, return false. Update `prev = node.val`.

B. Path Sum (Root to Leaf)

- **Pattern: Pre-order DFS**
- **Logic:** As you go down, subtract the node's value from the `targetSum`.
- **Tweak:** In the "Pre-order" spot (before calling children), check:

Java

```
if (node.left == null && node.right == null && targetSum - node.val == 0) ret
```

Then recurse: `return dfs(node.left, sum - val) || dfs(node.right, sum - val)`.

C. Flatten Binary Tree to Linked List

- **Pattern: Post-order DFS** (or reverse Pre-order)
- **Logic:** You need to move the left child to the right side.
- **Tweak:** You often process the right side first so you don't lose the reference to it, then attach it to the bottom of the left side.

D. Kth Smallest Element in a BST

- **Pattern: In-order DFS**
- **Logic:** Since In-order gives sorted values, simply run In-order traversal and decrement `k`.
- **Tweak:** In the "In-order" spot:

Java

```
k--;
if (k == 0) return node.val;
```

Pattern 3: Bottom-Up DFS (Post-Order Return)

1. Why do we use it?

We use this pattern when a node **cannot determine its own value** without first knowing the answers from its children.

- **Information Flow:** Information flows **UP** from the leaves to the root.
- **Decoupling:** The "Return Value" (what you pass to your parent) is often different from the "Final Answer" (what the problem asks for).
  - *Example:* To find the "Diameter" (width), a node must return its "Height" (depth) to its parent.

2. How to identify this pattern?

Look for problems that ask for some "maximum" or "aggregated" property that relies on subtree structure:

- "Find the diameter of the tree"
- "Find the maximum path sum"
- "Check if the tree is balanced"
- "Find the Lowest Common Ancestor (LCA)"
- "Longest Univalue Path"

3. The Core Mechanism

This uses **Post-order Traversal** logic (Children First), but with a twist: **The Global Variable Trick**.

1. **Base Case:** If `root == null`, return 0 (or a neutral value).
2. **Recursive Step:** Call the function on `left` and `right`. capture their return values.
3. **Calculation:** Use the children's values to compute the result for the *current* node.
4. **Global Update:** If the current node represents the best answer found so far (e.g., the widest diameter), update a global class-level variable.
5. **Return:** Return a value to the *parent* that helps the parent do *its* calculation (usually `1 + max(left, right)`).

4. Java Code Snippet (The Template)

Java

```java
public class Solution {
    // 1. GLOBAL VARIABLE to store the final answer
    // Because the recursive function returns "height", not the "answer".
    int globalMax = 0;

    public int solve(TreeNode root) {
        if (root == null) return 0;
        bottomUpDFS(root);
        return globalMax;
    }

    // This helper returns specific info to the PARENT (like Height)
    private int bottomUpDFS(TreeNode node) {
        // 2. Base Case
        if (node == null) return 0;

        // 3. Ask Children for Info (Divide)
        int leftVal = bottomUpDFS(node.left);
        int rightVal = bottomUpDFS(node.right);

        // 4. Logic for the ANSWER (The Global Max)
        // Example: Diameter = left height + right height
        int currentProblemResult = leftVal + rightVal;
        globalMax = Math.max(globalMax, currentProblemResult);

        // 5. Logic for the PARENT (The Return Value)
        // Example: My height = 1 + max(left, right)
        return 1 + Math.max(leftVal, rightVal);
    }
}
```

5. Example Questions & Logic Variations

A. Balanced Binary Tree

- **Goal:** A tree is balanced if the height difference between left and right subtrees of *every* node is ≤ 1.
- **The Trick:** Instead of a boolean, return the **Height**. If a subtree is unbalanced, return `-1` (an error code) to propagate the failure up.
- **Logic:**

Java

```java
int left = dfs(node.left);
int right = dfs(node.right);
if (left == -1 || right == -1 || Math.abs(left - right) > 1) return -1;
return 1 + Math.max(left, right);
```

B. Binary Tree Maximum Path Sum (Hard)

- **Goal:** Find the max path sum (can start and end anywhere).
- **Return to Parent:** `node.val + max(left, right)` (I can only extend a path from my parent to *one* of my children).
- **Global Update:** `node.val + left + right` (The path might curve through me, connecting my left and right children).
- **Crucial Edge Case:** If a child returns a negative value, ignore it (treat as 0), because adding a negative number hurts your sum.

C. Lowest Common Ancestor (LCA)

- **Goal:** Find the common ancestor of nodes `p` and `q`.
- **Logic:**
  - If I am `p` or `q`, return **myself**.
  - If `left` returns a node AND `right` returns a node, **I am the LCA** (return myself).
  - If only `left` returns something, pass that up.
  - If only `right` returns something, pass that up.

Pattern 4: Top-Down DFS (Path & Accumulator)

1. Why do we use it?

We use this pattern when the answer for a specific node depends on the **path it took to get there**. The node needs to know "What happened before I was reached?"

- **Accumulating Values:** You need to calculate a running total, a string sequence, or a list of nodes from the root down to the current leaf.
- **Path validity:** You need to check if the sequence of nodes from the root matches a specific pattern.

2. How to identify this pattern?

Look for problems that explicitly mention **"Root-to-Leaf"** or **"Path"**:

- "Find all paths that sum to X"
- "Return all root-to-leaf paths"
- "Sum of root-to-leaf numbers" (e.g., Path `1->2` represents the number `12`)
- "Check if a sequence exists in the tree"

3. The Core Mechanism

This pattern relies on **Passing Parameters Down**.

1. **Modified Signature:** Your recursive function will take extra arguments (the "Accumulator").
   - *Example:* `dfs(node, currentSum)` or `dfs(node, currentPathList)`.
2. **Process Current:** Before calling children, update the accumulator with the current node's value.
3. **Check Leaf:** If the current node is a leaf (no children), check if the accumulator satisfies the condition (e.g., `currentSum == target`).
4. **Recurse:** Pass the *updated* accumulator to `node.left` and `node.right`.
5. **Backtrack (Critical):** If you are passing a mutable object (like a `List`), you **must** remove the current node from the list before returning to the parent, so you don't pollute the path for the other sibling.

4. Java Code Snippet (The Template)

Java

```java
import java.util.*;

public class Solution {
    List<String> result = new ArrayList<>();

    public List<String> binaryTreePaths(TreeNode root) {
        if (root != null) topDownDFS(root, "");
        return result;
    }

    // The "pathSoFar" is the Accumulator passed DOWN
    private void topDownDFS(TreeNode node, String pathSoFar) {
        // 1. Update the accumulator (Process Current)
        String newPath = pathSoFar + node.val;

        // 2. Check Leaf Condition (Is this the end of a path?)
        if (node.left == null && node.right == null) {
            result.add(newPath); // We found a valid path!
            return;
        }

        // 3. Recurse Left and Right, passing the NEW history down
        if (node.left != null) {
            topDownDFS(node.left, newPath + "->");
        }
        if (node.right != null) {
            topDownDFS(node.right, newPath + "->");
        }
    }
}
```

Note on Backtracking: In the example above, `String` is immutable, so we didn't need to manually backtrack. If `pathSoFar` was a `List<Integer>`, we would need to do this:

Java

```
list.add(node.val);       // Add
dfs(node.left, list);     // Recurse
dfs(node.right, list);    // Recurse
list.remove(list.size() - 1); // BACKTRACK (Remove)
```

5. Example Questions & Logic Variations

A. Path Sum II (LeetCode 113)

- **Goal:** Return *all* paths that sum to a target.
- **Accumulator:** `List<Integer> currentPath`, `int currentSum`.
- **Leaf Logic:** If `currentSum == target`, add a *copy* of `currentPath` to the global result list.
- **Backtracking:** Required (add node, recurse, remove node).

B. Sum Root to Leaf Numbers

- **Goal:** Path `1 -> 2 -> 3` represents number `123`. Return sum of all such numbers.
- **Accumulator:** `int currentVal`.
- **Logic:**
  - Pass `currentVal * 10 + node.val` down to children.
  - If leaf, add this calculated value to a global `totalSum`.

C. Path With Given Sequence

- **Goal:** Check if the path matches an array `[1, 9, 2]`.
- **Accumulator:** `int index` (representing which index of the array we are matching).
- **Logic:**
  - If `node.val != array[index]`, return false immediately (pruning).
  - Recurse with `index + 1`.

Pattern 5: Tree Structure & Construction

1. Why do we use it?

We use this pattern when the problem involves creating a new tree from scratch or changing the shape of an existing tree.

- **Reconstruction:** Building a tree from flat arrays (e.g., "Pre-order" and "In-order" arrays).
- **Modification:** changing the left/right pointers to flip, flatten, or prune the tree.
- **Merging:** Combining two trees into one.

2. How to identify this pattern?

Look for verbs that imply creation or structural change:

- "Construct a binary tree from..."
- "Invert / Flip the binary tree"
- "Flatten the tree to a linked list"
- "Merge two binary trees"
- "Serialize and Deserialize"

3. The Core Mechanism

The key to this pattern is that your recursive function **returns a** `TreeNode`. Unlike previous patterns where we returned an `int` (height) or `void`, here the return value represents the **new root of the subtree** you just built or modified.

The "Pointer Reinforcement" Technique: You almost always assign the result of the recursive call back to the child pointer: `node.left = recursiveFunction(node.left);`

1. **Base Case:** If the input is empty/null, return `null`.
2. **Creation/Action:** Create a new node (or swap pointers for modification).
3. **Recursive Assignment:**
   - `node.left = solve(data for left side)`
   - `node.right = solve(data for right side)`
4. **Return:** Return `node` (the root) to the caller.

Java

```java
public class Solution {
    public TreeNode mergeTrees(TreeNode root1, TreeNode root2) {
        // 1. Base Cases: Handle nulls
        // If one is null, the "merged" result is just the other one.
        if (root1 == null) return root2;
        if (root2 == null) return root1;

        // 2. Action: Create/Update the node
        // Here, we sum the values of the two overlapping nodes
        TreeNode newNode = new TreeNode(root1.val + root2.val);

        // 3. Recursive Assignment (The Core Mechanism)
        // We trust the function to go off, build the left/right subtrees,
        // and bring back the pointer to the root of those subtrees.
        newNode.left = mergeTrees(root1.left, root2.left);
        newNode.right = mergeTrees(root1.right, root2.right);

        // 4. Return the result to the parent
        return newNode;
    }
}
```

5. Example Questions & Logic Variations

A. Invert Binary Tree (The Classic Google Question)

- **Goal:** Swap left and right children for every node.
- **Logic:**

Java

```
TreeNode temp = root.left;
root.left = invertTree(root.right); // Assign inverted right to left
root.right = invertTree(temp);      // Assign inverted left to right
return root;
```

B. Construct Binary Tree from Preorder and Inorder Traversal

- **Goal:** Rebuild the unique tree.
- **Logic:**
  - **Preorder** tells you the **Root** (always the first element).
  - **Inorder** tells you the **Structure** (everything left of the root value goes to the left child, everything right goes to the right child).
  - Use a `HashMap` to quickly find the index of the root value in the Inorder array to split the array into left/right halves.

C. Serialize and Deserialize Binary Tree

- **Goal:** Convert tree to string "1,2,X,X,3,X,X" (Pre-order) and back.
- **Logic (Deserialize):** Use a Queue (or global index). Read the first value. If it's "X" (null), return null. Otherwise, create a node, then recursively call `node.left = deserialize()` and `node.right = deserialize()`.

1. Why do we use it?

We use this pattern when the problem is about **projection**.

- **Side Views (Right/Left):** You are mapping the tree to a vertical line (the Y-axis). You only care about the "first" or "last" node you encounter at every depth.
- **Vertical Views (Top/Bottom):** You are mapping the tree to a horizontal line (the X-axis). You need to know which node stands "in front" of others at specific horizontal coordinates.

2. How to identify this pattern?

The questions are very literal:

- "Print the Right/Left view of the binary tree"
- "Print the Top/Bottom view"
- "Vertical Order Traversal"

3. The Core Mechanism

There are **two distinct sub-strategies** depending on the angle:

A. Side Views (Right or Left)

- **Strategy: DFS (Pre-order modification)** is the most elegant way.
- **Logic:** Keep track of the current `depth`. If you are visiting this `depth` for the first time, add the node to your result.
- **Trick:** For **Right** view, traverse `Right -> Left`. This ensures the first node you touch at any depth is the rightmost one. For **Left** view, traverse `Left -> Right`.

B. Vertical Views (Top or Bottom)

- **Strategy: Horizontal Distance (HD).**
- **Logic:** Assign coordinates to every node.
  - Root = 0
  - Left Child = `parent_HD - 1`
  - Right Child = `parent_HD + 1`
- Use a `TreeMap` or `HashMap` to store the mapping: `Map<HD, NodeValue>`.
  - **Top View:** Only add to the map if the HD key **does not exist** (first one seen from top).
  - **Bottom View:** Always **overwrite** the map value (last one seen is the bottom).

Java

```java
public class Solution {
    List<Integer> result = new ArrayList<>();

    public List<Integer> rightSideView(TreeNode root) {
        // Start DFS: depth 0
        solve(root, 0);
        return result;
    }

    public void solve(TreeNode node, int depth) {
        if (node == null) return;

        // 1. The "First Visit" Check
        // If the current depth equals the size of the result list,
        // it means we haven't added a node for this level yet.
        if (depth == result.size()) {
            result.add(node.val);
        }

        // 2. Traversal Order (CRITICAL)
        // Go RIGHT first so we see the right-side nodes first
        solve(node.right, depth + 1);
```

```
        // Then go LEFT (these will be skipped by the 'if' check above
        // if a right sibling already claimed the spot)
        solve(node.left, depth + 1);
    }
}
```

5. Example Questions & Logic Variations

A. Left Side View

- **Logic:** Same code as above, but swap the recursion order. Call `solve(node.left, ...)` first, then `solve(node.right, ...)`.

B. Top View (Vertical Logic)

- **Data Structure:** `Queue<Pair<Node, HD>>` and `Map<Integer, Integer>`.
- **Logic:** Use BFS.
  - When you push `node.left` to queue, give it `current_HD - 1`.
  - When you push `node.right` to queue, give it `current_HD + 1`.
  - If `!map.containsKey(current_HD)`, put `node.val` in the map.

C. Vertical Order Traversal (Harder)

- **Goal:** Return a list of columns `[[-2], [-1], [0,0,0], [1], [2]]`.
- **Logic:** Similar to Top View, but instead of a Map storing a *single* value, the Map stores a *List* of values for that HD. `Map<Integer, List<Integer>>`.
- *Note:* Strictly sorting the nodes within the same row/column often requires a custom comparator.

Pattern 7: Binary Search Tree (BST) Logic

1. Why do we use it?

We use this pattern to exploit the **sorted property** of the tree.

- **Property:** For every node, all values in the **Left** subtree are smaller, and all values in the **Right** subtree are larger.
- **Efficiency:** This allows us to make "decisions" at each node. Instead of exploring the whole tree, we can discard half of it at every step (similar to Binary Search in an array).

2. How to identify this pattern?

The problem statement will explicitly mention **"Binary Search Tree" (BST)**.

- "Find a value in a BST"
- "Find the Lowest Common Ancestor in a **BST**"
- "Trim the tree" or "Range Sum"
- "Convert a sorted array to a BST"

3. The Core Mechanism

The mechanism is **Pruning** (cutting off paths). Unlike standard DFS/BFS where you almost always call *both* `left` and `right`, here you use `if/else` logic to choose **one** direction (or conditionally both).

- If `target < node.val`: The answer *must* be in the **Left** child. (Ignore Right).
- If `target > node.val`: The answer *must* be in the **Right** child. (Ignore Left).
- If `target == node.val`: You found it!

4. Java Code Snippet (The Template)

Here is the standard **Search** template. It can be written recursively or iteratively.

Java

```java
public class Solution {
    public TreeNode searchBST(TreeNode root, int val) {
        // 1. Base Case: Not found or Found
        if (root == null || root.val == val) {
            return root;
        }

        // 2. Decision Logic (The "Pruning")
        if (val < root.val) {
            // Target is smaller, so it MUST be on the left.
            // We completely abandon the right branch.
            return searchBST(root.left, val);
        } else {
            // Target is larger, so it MUST be on the right.
            return searchBST(root.right, val);
        }
    }
}
```

5. Example Questions & Logic Variations

A. Range Sum of BST

- **Goal:** Sum all values between `[low, high]`.
- **Logic (Pruning):**
  - If `node.val < low`: The current node is too small. **Go Right** (to find bigger numbers). *Don't go Left.*
  - If `node.val > high`: The current node is too big. **Go Left** (to find smaller numbers). *Don't go Right.*
  - If `low <= node.val <= high`: Include this node, and explore **both** children.

B. Lowest Common Ancestor (LCA) of a BST

- **Goal:** Find the split point where `p` and `q` diverge.
- **Logic:**
  - If both `p` and `q` are smaller than root: **Go Left**.
  - If both `p` and `q` are larger than root: **Go Right**.
  - Otherwise (one is smaller, one is larger, or one is the root): **Current node is the LCA.** (This is much simpler than the LCA for a standard binary tree).

C. Convert Sorted Array to Binary Search Tree

- **Goal:** Build a height-balanced BST.
- **Logic (Divide & Conquer):**
  - Pick the **middle** element of the array. Make it the `root`.
  - Recursively build the left child using the *left half* of the array.
  - Recursively build the right child using the *right half* of the array.

Summary Checklist for Interviews

You now have the complete toolkit for Tree questions. When you see a problem, run this mental checklist:

1. **Level by level? → BFS (Queue)**
2. **Path finding / Backtracking? → Top-Down DFS**
3. **Tree Construction / Modification? → Structure Pattern (Pointer Reinforcement)**
4. **Information from leaves (Diameter, Height)? → Bottom-Up DFS (Post-order)**
5. **Looking from the side? → Tree View (DFS with depth check)**
6. **Is it a BST? → Search Logic (Pruning)**

In coding interviews, "TLE" (Time Limit Exceeded) usually happens if your Time Complexity is worse than the expected solution, or if you cause a Stack Overflow by ignoring Space Complexity constraints.

Key Definitions for this guide:

- N: Total number of nodes in the tree.
- H: Height of the tree.
  - **Balanced Tree:** $H \approx \log N$
  - **Skewed Tree (Worst Case):** $H \approx N$ (looks like a linked list).
- W: Maximum Width (max nodes in a single level).
  - **Balanced Tree:** $W \approx N/2$

1. Breadth-First Search (BFS) / Level Order

- **Time Complexity:** $O(N)$
  - **Why:** You process every node exactly once.
  - **TLE Risk:** Minimal. BFS is very stable.
- **Space Complexity:** $O(W)$ (Queue size)
  - **Why:** In the worst case (a perfect binary tree), the bottom-most level contains roughly $N/2$ nodes. Your queue will hold all of them at once.
  - **Note:** This is usually **more** memory-intensive than DFS for balanced trees.

2. Depth-First Search (DFS - Pre/In/Post)

- **Time Complexity:** $O(N)$
  - **Why:** You visit every node exactly once.
- **Space Complexity:** $O(H)$ (Recursion Stack)
  - **Balanced Tree:** $O(\log N)$
  - **Skewed Tree:** $O(N)$
  - **TLE/Error Risk:** If the tree is extremely deep (skewed) and $N > 10,000$, basic recursion might cause a **Stack Overflow Error** (which is a form of space TLE).

3. Bottom-Up DFS (Post-Order Return)

- **Time Complexity:** $O(N)$
  - **Why:** Even though information flows up, you still only visit each node once.
  - **TLE Risk:** A common mistake is calculating the height repeatedly for every node (e.g., calling `getHeight(node.left)` inside a loop). This degrades time to $O(N^2)$. **Always** compute height once and pass it up to avoid this.
- **Space Complexity:** $O(H)$ (Recursion Stack)

4. Top-Down DFS (Path & Accumulator)

- **Time Complexity:** $O(N)$
  - **Hidden Cost:** String concatenation (e.g., `path + "->" + val`) inside the recursion can turn this into $O(N^2)$ because string creation is expensive (it copies characters).
  - **Fix:** Use `StringBuilder` (Java) or maintain a `List` and backtrack to keep it $O(N)$.
- **Space Complexity:** $O(H)$
  - If you are storing *all* paths for the final answer, the space can grow to $O(N \cdot H)$ in the worst case.

5. Tree Structure & Construction

- **Time Complexity:** $O(N)$
  - **Why:** You usually iterate through an array (like Pre-order array) and create one node per element.
- **Space Complexity:** $O(N)$
  - **Why:** You are creating N new objects to build the new tree. plus $O(H)$ for the recursion stack.

6. Tree View Patterns (Right/Left/Top View)

- **Time Complexity:** $O(N)$
  - **Why:** You must traverse the whole tree to know which node is "visible" at the deepest level.
  - **Sorting Cost:** If using "Vertical Order Traversal" where you sort nodes by value within the same column, complexity becomes $O(N \log N)$.
- **Space Complexity:** $O(N)$
  - **Why:** For Top/Bottom view, you often store a Map containing one entry per vertical column. In the worst case, you store nearly all nodes.

7. BST Specific Patterns (Search/Pruning)

- **Time Complexity:** $O(H)$
  - **Balanced BST:** $O(\log N)$ (Great!)
  - **Skewed BST:** $O(N)$ (Bad!)
  - **Why:** You discard half the tree at every step.
- **Space Complexity:** $O(H)$
  - **Why:** Recursion stack depth equals the height. Iterative solutions can reduce this to $O(1)$ space.

Summary "Cheat Sheet"

Pattern	Time Complexity	Space Complexity	Best For...
1. BFS (Level Order)	$O(N)$	$O(Width)$	Shortest path, Level averages
2. DFS (Standard)	$O(N)$	$O(Height)$	Exhaustive search, Serializing
3. Bottom-Up DFS	$O(N)$	$O(Height)$	Diameter, Balanced check, LCA
4. Top-Down DFS	$O(N)$	$O(Height)$	Path Sums, Sequence matching
5. Construction	$O(N)$	$O(N)$	Building trees, Inverting
6. Tree Views	$O(N)$	$O(N)$	Right/Left/Top Views
7. BST Search	$O(H)$	$O(H)$ or $O(1)$	Range sum, Finding values